

Bellerophon Language Documentation

Language Overview

Bellerophon is a domain-specific scripting language (DSL) for 3D printing. Write clean, procedural motions and parametric macros that compile directly to G-code for Klipper or Marlin firmware.

Note:

Philosophy: Precise, readable commands that make controlling 3D printers easier.

Getting Started & Platform Requirements

Bellerophon is a 3D printer scripting IDE and language designed for Klipper and Marlin macros. It allows you to write structured, parametric scripts and compile them into firmware-compatible G-code macros. This section describes system requirements, inputs, outputs, capabilities, and limitations.

1. Requirements

- **Software:** Java Runtime Environment (JRE) 21 or newer
- **Hardware:** Any computer capable of running Java

2. Running the IDE

Windows

1. Double-click `run.bat` to start the server
2. Your browser will open automatically to the Dimidium Ecosystem
3. Close the server window when finished

Linux / Mac

1. Install Java (OpenJDK 21 or newer)
2. Open a terminal in the SyntaxN folder
3. Run: `java -jar SyntaxN.jar`
4. Open browser to: `http://localhost:4567`
5. Press Ctrl+C in terminal to stop the server

Note:

The server runs locally on port 4567. No internet connection required after installation.

3. Compilation Targets: Klipper vs Marlin

The IDE can compile your .bph scripts to two different output formats:

- **Klipper** – Full support for all Bellerophon features, including `if` statements, advanced temperature commands, and Klipper-specific G-code macros.
- **Marlin** – A simpler output that omits features not available in Marlin firmware. **Conditional statements (`if/endif`) are not supported in Marlin mode**; all other movement, heating, looping, and macro commands work as expected (loops are expanded at compile time).

Select the target in the IDE before compiling. The generated .cfg (Klipper) or .gcode (Marlin) file will be saved accordingly.

4. Inputs

- Bellerophon accepts only .bph script files.
- The IDE provides real-time syntax validation, highlighting errors as you type.
- Scripts are line-oriented: each command must be on its own line; blank lines between commands are ignored.
- Supported commands include: movement, temperature control, heaters, fans, waits, pauses, macros calls, repeat loops, Brepeat loops, conditional execution (Klipper only), extruder control, and bed/probe operations.

5. Outputs

- Compiles .bph scripts into either Klipper macro .cfg files or Marlin .gcode files.
- Compiled macros can be deployed to printers via **Gravity Hub** or directly from Mainsail (Klipper) or any Marlin host.
- The IDE provides Build Log and Exceptions tabs for debugging and catching compilation errors.

6. Capabilities

- Compile structured macros for Klipper or Marlin with correct syntax.
- Support for arithmetic expressions in coordinates and parameters: **globally available**
- Mathematical functions: `sin`, `cos`, `tan`, `sqrt`, `abs` (absolute value), `sign` (signum), power operator `^`, and constant `pi`.
- Parametric loops using `repeat` (static repetition) and `Brepeat` (dynamic, iteration-aware loops), including support for **nested loops** and iterator-based expressions.
- **Parametric Geometry**: Mathematical functions allow the generation of procedural motion paths such as circles, spirals, or arbitrary curves.
- Conditional execution with `if` statements (Klipper only), allowing decisions based on temperatures or other parameters.
- Temperature management for extruder, bed, and chamber (`Set_Heater_Temperature`, `Heat`, `WaitForTemp`).
- Printer movement commands, including absolute and relative positioning, X/Y/Z moves, extruder control, and dwell/wait commands.
- Macro calls within macros (`M.call`), allowing reuse and modular scripts.
- Bed and probe management: `BED_MESH_CALIBRATE`, `PROBE_CALIBRATE`, `LoadBedMesh`.
- Fan control, pressure advance, speed settings, and idle timeout (`TIMEOUT_SET`).
- Pause, resume, cooldown, and SD card file printing (`PRINTFILE`).

- **IDE Reference Panel:** Displays all Bellerophon tokens in a user-friendly format. Users can pin commands to a sidebar for quick access while typing, reducing errors and speeding up workflow.
- **Session Restoration:** The IDE automatically restores the previous editing session to prevent accidental data loss and allow users to continue work seamlessly.
- **Dimidium Ecosystem Integration:** Compiled macros can be deployed and managed through the Dimidium Gravity Hub dashboard or executed directly within Mainsail (Klipper) or any Marlin host.

7. Limitations

- Cannot slice 3D models; only generates G-code from Bellerophon .bph scripts.
- Does not modify existing G-code files automatically.
- Some advanced or rare G-code commands may not be supported.
- No general-purpose variables; logic is limited to loop iterators and math expressions
- **Marlin target:** Conditional statements (if/endif) are not supported and will be ignored

9. Workflow Diagram

High-level workflow of using Bellerophon:

.bph script → IDE (edit & validate) → Select target → Compile → .cfg (Klipper) / .gcode (Marlin) → Deploy → Run macros

Program Structure

A Bellerophon program consists of one or more macros. Each macro starts with `M.title` and ends with `M.end`.

```
M.title "Macro Name"

    [statement 1]

    [statement 2]

    [statement 3]

M.end
```

Each command must be on its own line. Indentation is optional but recommended. Comments start with `#`.

Statement Reference

COMMAND	SYNTAX	EXAMPLE	G-CODE / NOTES
Home	Home OF Home -> coordList	Home OF Home -> x=0 y=0 z=0	G28 / G28 X Y Z

COMMAND	SYNTAX	EXAMPLE	G-CODE / NOTES
<code>Move</code>	<code>Move direction</code>	<code>Move left / right / center / up / down</code>	G1 with directional offset. Iterates 1 mm. center defaults to X0 Y0
<code>MoveTo</code>	<code>MoveTo coordList</code>	<code>MoveTo x=100 y=100 z=0.2</code>	G1 X Y Z E
<code>Absolute</code>	<code>Absolute</code>	<code>Absolute</code>	G90
<code>Relative</code>	<code>Relative</code>	<code>Relative</code>	G91
<code>Heat</code>	<code>Heat target=number</code>	<code>Heat extruder=200</code>	Sets temperature and waits until it is reached . Extruder → M109, Bed → M190, Chamber → M141
<code>Set_Heater_Temperature</code>	<code>Set_Heater_Temperature target=number</code>	<code>Set_Heater_Temperature bed=60</code>	Sets temperature but does NOT wait . Extruder → M104, Bed → M140, Chamber → M141
<code>WaitForTemp</code>	<code>WaitForTemp target</code>	<code>WaitForTemp extruder</code>	Waits until the target reaches its current set temperature. Does not change the temperature. Extruder → M109, Bed → M190, Chamber → M141
<code>Cooldown</code>	<code>Cooldown target (optional)</code>	<code>Cooldown extruder</code> or just <code>Cooldown</code>	M104 S0 / M140 S0 / TURN_OFF_HEATERS. If target omitted, turns off all heaters.
<code>TimeoutSet</code>	<code>TIMEOUT_SET seconds</code>	<code>TIMEOUT_SET 300</code>	Set idle timeout (seconds). Maps to Klipper's SET_IDLE_TIMEOUT TIMEOUT=300 (ignored in Marlin).
<code>Pause</code>	<code>Pause</code>	<code>Pause</code>	PAUSE (Klipper) / M0 (Marlin)
<code>Resume</code>	<code>Resume</code>	<code>Resume</code>	RESUME (Klipper) / M108 (Marlin)
<code>Respond</code>	<code>Respond MSG "text"</code>	<code>Respond MSG "Print complete"</code>	RESPOND MSG="text" (Klipper) / M117 (Marlin)
<code>SetSpeed</code>	<code>SetSpeed = number</code>	<code>SetSpeed = 100</code>	G1 F#
<code>SetFan</code>	<code>SetFan = number</code>	<code>SetFan = 255</code>	SET_FAN SPEED=# (Klipper) / M106 S# (Marlin)
<code>PrintFile</code>	<code>PRINTFILE "file.gcode"</code>	<code>PRINTFILE "cube.gcode"</code>	SDCARD_PRINT_FILE FILENAME="file.gcode" (Klipper) / M23 (Marlin)
<code>Call Macro</code>	<code>M.call "MacroName"</code>	<code>M.call "CalibrateBed"</code>	Calls another macro (Klipper) / M98 (Marlin)
<code>If / EndIf</code>	<code>if condition ... endif</code>	<code>if extruder > 190 ... endif</code>	Klipper only ; compiled to Jinja condition. Not available in Marlin target.
<code>RelativeExtrusion</code>	<code>RelativeExtrusion</code>	<code>RelativeExtrusion</code>	M83
<code>ResetExtruder</code>	<code>ResetExtruder</code>	<code>ResetExtruder</code>	G92 E0

COMMAND	SYNTAX	EXAMPLE	G-CODE / NOTES
BedMeshCalibrate	BedMeshCalibrate	BedMeshCalibrate	BED_MESH_CALIBRATE (Klipper) / G29 (Marlin)
Level	Level	Level	Alias for BedMeshCalibrate → triggers BED_MESH_CALIBRATE or G29
ProbeCalibrate	ProbeCalibrate	ProbeCalibrate	PROBE_CALIBRATE (Klipper) – not available in Marlin
LoadBedMesh	LoadBedMesh "profile"	LoadBedMesh "default"	BED_MESH_PROFILE LOAD=profile (Klipper) – not available in Marlin
SetPressureAdvance	SetPressureAdvance number	SetPressureAdvance 0.04	SET_PRESSURE_ADVANCE ADVANCE=# (Klipper only)
Dwell	Dwell time S ms (optional unit)	Dwell 0.5 S or Dwell 500 ms	G4 P# (P is milliseconds). If unit omitted, defaults to seconds. Example: Dwell 0.5 → G4 P500

Loops & Arithmetic Operators

Bellerophon supports two types of loops: `repeat` for static repetition and `Brepeat` for dynamic loops.

Note:

Positioning Context in Loops: The behavior of movement inside a loop depends heavily on the active mode:

- **Relative (G91):** Coordinates are calculated as offsets from the last position. Each move builds on the previous one. Best for iteratively forming shapes, spirals, zig-zags, or any flowing patterns.
- **Absolute (G90):** Coordinates are calculated from a fixed origin on the bed. Each move goes to a specific point. Use this only when you need precise positions that don't depend on the previous move, like moving to a center point or fixed markers.

Example: Absolute vs Relative in Loops

The same loop code produces very different results depending on the active mode:

```
# Absolute Mode Example

Absolute

MoveTo x=10

Brepeat 4

    MoveTo x+=5

end
```

Relative Mode Example

Relative

MoveTo x=10

Brepeat 4

 MoveTo x+=5

end

Output:

Absolute Mode Output

[gcode_macro Absolute!-tester]

gcode:

G90

G1 X10.000

G1 X15.000

G1 X20.000

G1 X25.000

G1 X30.000

Relative Mode Output

[gcode_macro Relative!-tester]

gcode:

G91

G1 X10.000

G1 X5.000

G1 X5.000

G1 X5.000

G1 X5.000

Note:

Key observation: In absolute mode, `x+=5` calculates a new absolute position (10+5=15, then 15+5=20, etc.). In relative mode, it adds 5 to the current position each time, creating equal-sized steps from wherever the previous move ended.

Assignment Operators

Operators allow for precise control during movement:



= Direct Assignment

- += Add to current
- -= Subtract from current
- *= Multiply current
- /= Divide current

1. Static Repeat (`repeat`)

The `repeat` command simply duplicates the enclosed G-code commands a specified number of times during compilation.

```
repeat 3

    MoveTo x+=10 y+=10

    Respond MSG "Moving..."

end
```

Output (repeat):

```
G1 X10.0 Y10.0

RESPOND MSG="Moving..."

G1 X20.0 Y20.0

RESPOND MSG="Moving..."

G1 X30.0 Y30.0

RESPOND MSG="Moving..."
```

Note:

The commands are physically duplicated in the output G-code. Great for simple, fixed repetitions.

2. Dynamic Loop (`Brepeat`)

The `Brepeat` command creates a loop that supports arithmetic expressions and mathematical functions. Unlike `repeat`, this loop exposes an iterator variable `i` which increments each iteration starting from 0. Expressions are evaluated during compilation, producing numeric G-code values.

Note:

Important: The iterator `i` is only valid inside `Brepeat` loops, but **mathematical functions (sin, cos, tan, sqrt, abs, sign) and the power operator (^) are globally available anywhere an expression is allowed** (e.g., in MoveTo, Heat, SetSpeed, etc.).

Example: Using the iterator

```
Brepeat 5  
    MoveTo x=i  
end
```

Example: Mathematical Expressions (global availability)

Bellerophon supports arithmetic expressions anywhere: including exponentiation `^`, absolute value `abs()`, signum `sign()`, and the constant `pi`.

```
Brepeat 3  
    MoveTo x=i^2 * 5 + abs(i-2)  
end  
  
Heat extruder=180 + 20*sin(3.14159/2)  
# works outside loops too
```

Example: Drawing a Square (Relative)

Using **Relative** mode ensures each move is from the previous point, naturally forming shapes when iterating.

```
Relative  
  
Brepeat 4  
    MoveTo x=10 y=0  
    MoveTo x=0 y=10  
    MoveTo x=-10 y=0  
    MoveTo x=0 y=-10  
end
```

Example: Drawing a Circle (Relative)

Using **Relative** mode allows the circle to be built incrementally, keeping each move relative to the last point for smooth motion.

```
Relative  
  
Brepeat 60  
    MoveTo x=cos(i*0.1)*5 y=sin(i*0.1)*5  
end
```

Example: Drawing a Spiral (Relative)

Using **Relative** mode allows complex parametric shapes to be built incrementally. This spiral uses the loop counter i to calculate both angle and radius, creating an expanding spiral pattern.

```
# Spiral test macro

M.title "spiral_test"

MoveTo x = 160 y = 160 z= 10

Relative

SetSpeed = 6500

Brepeat 65

    MoveTo x=cos(i*10)*(i*0.1) y=sin(i*10)*(i*0.1)

end

Dwell 2 ms

Absolute

MoveTo x = 150 y = 150

Dwell 2 s

Home

M.end
```

Output:

```
[gcode_macro spiral_test]

gcode:

    G90

    G1 X160.000 Y160.000 Z10.000

    G91

    G1 F6500

    G1 X0.000 Y0.000

    G1 X0.098 Y0.017

    G1 X0.188 Y0.068

    G1 X0.260 Y0.150

    G1 X0.306 Y0.257

    G1 X0.321 Y0.383

    G1 X0.300 Y0.520

    G1 X0.239 Y0.658

    G1 X0.139 Y0.788

    G1 X0.000 Y0.900

    G1 X-0.174 Y0.985
```

G1 X-0.376 Y1.034

G1 X-0.600 Y1.039

G1 X-0.836 Y0.996

G1 X-1.072 Y0.900

G1 X-1.299 Y0.750

G1 X-1.504 Y0.547

G1 X-1.674 Y0.295

G1 X-1.800 Y0.000

G1 X-1.871 Y-0.330

G1 X-1.879 Y-0.684

G1 X-1.819 Y-1.050

G1 X-1.685 Y-1.414

G1 X-1.478 Y-1.762

G1 X-1.200 Y-2.078

G1 X-0.855 Y-2.349

G1 X-0.451 Y-2.561

G1 X-0.000 Y-2.700

G1 X0.486 Y-2.757

G1 X0.992 Y-2.725

G1 X1.500 Y-2.598

G1 X1.993 Y-2.375

G1 X2.451 Y-2.057

G1 X2.858 Y-1.650

G1 X3.195 Y-1.163

G1 X3.447 Y-0.608

G1 X3.600 Y-0.000

G1 X3.644 Y0.642

G1 X3.571 Y1.300

G1 X3.377 Y1.950

G1 X3.064 Y2.571

G1 X2.635 Y3.141

G1 X2.100 Y3.637

G1 X1.471 Y4.041

G1 X0.764 Y4.333

G1 X0.000 Y4.500

G1 X-0.799 Y4.530

G1 X-1.607 Y4.417

G1 X-2.400 Y4.157

G1 X-3.150 Y3.754

G1 X-3.830 Y3.214

G1 X-4.417 Y2.550

G1 X-4.886 Y1.779

G1 X-5.219 Y0.920

G1 X-5.400 Y0.000

G1 X-5.416 Y-0.955

G1 X-5.262 Y-1.915

G1 X-4.936 Y-2.850

G1 X-4.443 Y-3.728

G1 X-3.792 Y-4.520

G1 X-3.000 Y-5.196

G1 X-2.086 Y-5.732

G1 X-1.077 Y-6.106

G1 X-0.000 Y-6.300

G1 X1.111 Y-6.303

G1 X2.223 Y-6.108

G1 X3.300 Y-5.716

G1 X4.307 Y-5.132

G1 X5.209 Y-4.371

G1 X5.976 Y-3.450

G1 X6.578 Y-2.394

G1 X6.992 Y-1.233

G1 X7.200 Y-0.000

G1 X7.189 Y1.268

G1 X6.954 Y2.531

G1 X6.495 Y3.750

G1 X5.822 Y4.885

G1 X4.949 Y5.899

G1 X3.900 Y6.755

G1 X2.702 Y7.424

G1 X1.389 Y7.878

G1 X0.000 Y8.100

G1 X-1.424 Y8.075

G1 X-2.839 Y7.799

G1 X-4.200 Y7.275

G1 X-5.464 Y6.511

G1 X-6.588 Y5.528

G1 X-7.534 Y4.350

G1 X-8.269 Y3.010

G1 X-8.765 Y1.545

G1 X-9.000 Y0.000

G1 X-8.962 Y-1.580

G1 X-8.645 Y-3.147

G1 X-8.054 Y-4.650

G1 X-7.201 Y-6.042

G1 X-6.106 Y-7.277

G1 X-4.800 Y-8.314

G1 X-3.318 Y-9.115

G1 X-1.702 Y-9.651

G1 X0.000 Y-9.900

G1 X1.736 Y-9.848

G1 X3.454 Y-9.491

G1 X5.100 Y-8.833

G1 X6.621 Y-7.890

G1 X7.967 Y-6.685

G1 X9.093 Y-5.250

G1 X9.961 Y-3.625

G1 X10.537 Y-1.858

G1 X10.800 Y-0.000

G1 X10.734 Y1.893

G1 X10.337 Y3.762

G1 X9.613 Y5.550

G1 X8.580 Y7.199

G1 X7.263 Y8.656

G1 X5.700 Y9.873

G1 X3.933 Y10.806

G1 X2.014 Y11.424

G1 X-0.000 Y11.700

G1 X-2.049 Y11.621

G1 X-4.070 Y11.182

G1 X-6.000 Y10.392

G1 X-7.778 Y9.269

G1 X-9.346 Y7.842

G1 X-10.652 Y6.150

G1 X-11.652 Y4.241

G1 X-12.310 Y2.171

G1 X-12.600 Y0.000

G1 X-12.507 Y-2.205

G1 X-12.028 Y-4.378

G1 X-11.172 Y-6.450

G1 X-9.959 Y-8.356

G1 X-8.421 Y-10.035

G1 X-6.600 Y-11.432

G1 X-4.549 Y-12.498

G1 X-2.327 Y-13.196

G1 X0.000 Y-13.500

G1 X2.362 Y-13.393

G1 X4.686 Y-12.874

G1 X6.900 Y-11.951

G1 X8.935 Y-10.648

G1 X10.725 Y-8.999

G1 X12.211 Y-7.050

G1 X13.344 Y-4.857

G1 X14.083 Y-2.483

G1 X14.400 Y-0.000

G1 X14.280 Y2.518

G1 X13.720 Y4.993

G1 X12.731 Y7.350

G1 X11.337 Y9.513

G1 X9.578 Y11.414

G90

G28

Note:

How the spiral formula works: The expression `cos(i*10)*(i*0.1)` and `sin(i*10)*(i*0.1)` create an expanding spiral where:

- `i*10` controls the angular step (angle increases quickly, making many turns)
- `i*0.1` controls the radius growth (spiral expands outward slowly)
- The product creates a spiral that grows in radius as it rotates

- Using **Relative** mode means each move is an offset from the previous position

Example Program

```
M.title "Bed Leveling Routine"

Absolute

SetSpeed = 3000

Home -> x=0 y=0

MoveTo z=5

Heat bed=60

Respond MSG "Bed heating complete"

Level

LoadBedMesh "default"

MoveTo x=100 y=100

ProbeCalibrate

SetPressureAdvance 0.04

ResetExtruder

RelativeExtrusion

Move right

Move up

Pause

Resume

Cooldown

M.end
```